

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JNANA SANGAMA”, BELAGAVI, KARNATAKA – 590018



An Internship Report on

“ANDROID APP DEVELOPMENT USING GENAI”

*Submitted in partial fulfilment for the award of the degree of Bachelor of Engineering in
Computer Science and Engineering during the year 2025 – 2026*

Submitted By

4MN22CS026

Maria Saji

Under the guidance of

Internal Guide

Prof. Poornima H
Assistant Professor
Department of CS&E
MITT, Mysore



External Guide

Mr. Tirumal Mutalikdesai
Program Coordinator
MindMatrix
Bangalore



2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(NBA Accredited for the Academic Year 2024 – 2027)

MAHARAJA INSTITUTE OF TECHNOLOGY THANDAVAPURA

Just Off NH 766, Nanjanagudu Taluk, Mysore District – 571302

(Approved by AICTE, accredited by NBA, New Delhi and Affiliated by VTU, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(NBA Accredited for the Academic Year 2024 – 2027)
MAHARAJA INSTITUTE OF TECHNOLOGY THANDAVAPURA
MYSORE – 571302



CERTIFICATE

This is to certify that the Internship work titled “*Android App development using GenAi*” has been successfully carried out by **MARIA SAJI [4MN22CS026]** Bonafide student of Maharaja Institute of Technology Thandavapura in partial fulfilment of the requirements for the Degree of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi during the academic year 2025-26. The Internship report has been approved as it satisfies the academic requirements for the Internship work prescribed for the Bachelor of Engineering Degree.

Internal Guide
Prof. Poornima H
Assistant Professor
Dept. of CS&E, MITT

External Guide
Mr. Tirumal Mutalikdesai
Program Coordinator
MindMatrix

HoD
Dr. Ranjit K N
Professor and Head
Dept. of CS&E, MITT

Principal
Dr. Y T Krishne Gowda
Principal, MITT

External Viva

Name of the Examiners

Signature with Date

1. _____
2. _____

DECLARATION

We hereby declare that the Internship work entitled “Android App Development Using GenAi” submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering to Visvesvaraya Technological University, Belagavi, is a record of our original work carried out at **MindMatrix** under the supervision of Internal Guide [**Prof. Poornima H, Assistant Professor, Department of CS&E**] and External Guide [**Mr. Tirumal Mutalikdesai, Program Coordinator, MindMatrix**] during the academic year **2025–26**. This industry internship dissertation report is based on the professional work, projects, and assignments undertaken at **MindMatrix**, reflecting the practical exposure and technical contributions made during the internship period. The report is submitted in partial fulfillment for the award of the Bachelor’s degree in Computer Science and Engineering. We further declare that the results and findings embodied in this report are derived from the company internship work and have not been submitted to any other University or Institute for the award of any degree.

MARIA SAJI

[4MN22CS026]

Date: 07/05/2026

Place: Mysuru

ACKNOWLEDGEMENT

It gives us immense pleasure to bring our Internship report entitled “*Android App Development using GenAi9*” in the final year engineering course.

We are very thankful to **Dr. Y T Krishne Gowda, Principal, MITT** for having supported us in our academic endeavors.

We would like to extend our heartfelt thanks to **Dr. Ranjit K N, Professor and Head, Dept. of CS&E**, for providing us timely suggestions, encouragement, and support to complete this work.

We would like to express our sincere gratitude to the Internship Internal Guide **Prof. Poornima H, Assistant Professor, Dept. of CS&E** for his valuable support, suggestions, and coordination throughout this Internship.

We would also like to thank all the teaching and non-teaching staff members of the department for their support. We are always thankful to our parents for their valuable support and guidance in every step.

We express our deepest gratitude and indebt thanks to MITT which has provided us an opportunity in fulfilling our most cherished desire of reaching the goal.

ABSTRACT

The internship was undertaken at MindMatrixEd, an e-learning platform based in Bengaluru, Karnataka, as part of the Android App Development using GenAI programme. The programme spanned 15 weeks, commencing 2nd February 2026, and was conducted in online (remote) mode via the MindMatrix Learning Management System and the Discord platform VTU-Frontiers-Official. The internship involved progressive project work beginning with foundational Android concepts and advancing to a production-level healthcare application titled Matru-Sneh Health. This application is designed as a pocket pregnancy guide for rural expectant mothers, featuring a debounce-protected kick counter, check-up countdown with WorkManager reminders, a daily nutrition plate checklist, and a full-screen danger sign health alert system. The application was developed following MVVM architecture with a five-table Room database for offline-first data persistence. Additionally, a side project titled Place Explorer, a tourist attraction finder with budget estimation and itinerary planning was developed. Across both projects, AI-assisted development tools including ChatGPT and Antigravity were integrated for code generation, debugging, and UI design. The internship resulted in measurable technical skill development in Kotlin, Android Studio, GitHub, Room DB, WorkManager, and Jetpack architecture components.

CONTENTS

Sl. No.	Contents	Page No.
1	COMPANY PROFILE	[1-2]
1.1	Overview of the Company	1
1.2	Services offered by the Company	1-2
2	INDUSTRY INTERNSHIP PROGRAMME	[3-6]
2.1	Introduction	3
2.2	Purpose of Internship	3-4
2.3	Importance of Internship	4-5
2.4	Roles and Responsibilities	5-6
3	INTERNSHIP OBJECTIVES	[6-8]
3.1	Learning Goals	6
3.2	Professional Growth Areas	6-7
3.3	Internship Schedule	7-8
4	TOOLS AND TECHNOLOGIES USED	[9-11]
4.1	Software	9-10
4.2	Platforms	10
4.3	Tools	10-11
5	WORK DESCRIPTION	[13-20]
5.1	Methodology	12-13
5.2	System Design / architecture	13-15
5.3	Modules	16-17
5.4	Implementation Details	17-20
6	RESULTS & DISCUSSION	[21-23]
6.1	Output and Screenshots	21
6.2	Performance Metrics	22
6.3	Evaluation and Analysis	22-23
	CONCLUSION AND FUTURE ENHANCEMENT	[24-25]
	BIBLIOGRAPHY	26
	APPENDIX	27

LIST OF FIGURES

Figure No.	Figure Title	Page No.
5.1	MVVM Architecture Layer Overview	15
5.4	Room Database Schema	19
6.1(a)	Place Explorer App (Project 1)	21
6.2(b)	Matru-Sneh Health (Main Project)	21

LIST OF TABLES

Table No.	Table Title	Page No.
3.3	Internship Weekly Breakdown	7
4.1	Technologies and Tools Used	9
4.3	Tools Used	10-11
5.2	Room Database Schema	13
5.3	Modules	16
5.4	Database Schema	18-19
6.2	Performance Metrics	22
6.3	Evaluation and Analysis	22-23

LIST OF ACRONYMS

Figure No.	Figure Title	Page No.
GenAI	Generative Artificial Intelligence	1
IDE	Integrated Development Environment	10
LMS	Learning Management System	10
MVVM	Model-View-ViewModel	15
PRD	Product Requirements Document	5

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

VISION OF THE INSTITUTION

To be recognized as a premier institute in creating competent graduates driven towards socio-technical needs

MISSION OF THE INSTITUTION

- M 1:** To exhibit quality in the processes of teaching and learning evolved through continual feedback.
- M 2:** To create an ecosystem of greater learning through research and innovation.
- M 3:** To engage in self-learning through interaction with industry and alumni.
- M 4:** To maintain professional and ethical approach in dealing with stakeholders.

VISION OF THE DEPARTMENT

To be a forerunner in producing skilled technocrats in the field of computing for the betterment of the society

MISSION OF THE DEPARTMENT

- M 1:** To impart knowledge of logical, sequential and procedural implications of computing technology.
- M 2:** To expose recent advances in computing technology and plan for upgradation attitude in enhancing skills.
- M 3:** To create innovators having apt professional aptitude with the sense of engineering ethics.

PROGRAMME SPECIFIC OUTCOMES (PSOs)

On completion of B.E Computer Science & Engineering Program, The graduates will be able to -

- PSO 1:** Develop and maintain existing computing systems.
- PSO 2:** Create innovative career paths to be an entrepreneur or through higher studies.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

In support of the mission, within few years of graduation, the Computer Science & Engineering programme will enable its graduates to -

- PEO 1:** Attain a career in industry and be successful via adapting to fast-changing technologies through self and incessant learning.
- PEO 2:** Pursue excellence in pioneering newer technologies by being a part of the evolving challenges.
- PEO 3:** Evaluate optimal allocation of resources either in execution of job or as an entrepreneur.

PROGRAM OUTCOMES (POs)

- PO 1: Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- PO 2: Problem analysis:** Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- PO 3: Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO 4: Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- PO 5: Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
- PO 6: The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- PO 7: Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- PO 8: Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- PO 9: Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- PO 10: Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- PO 11: Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- PO 12: Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

CHAPTER – 1

COMPANY PROFILE

1.1 Overview of the Company

MindMatrix, operating its educational wing as MindMatrixEd, is an e-learning platform headquartered in Bengaluru, Karnataka, India. The organization delivers industry-aligned training programmes specifically designed for engineering students, bridging the gap between academic learning and real-world industry expectations. MindMatrixEd provides a structured learning environment through its proprietary Learning Management System accessible at lms.mindmatrix.io, and facilitates collaborative peer interaction via its Discord server, VTU-Frontiers-Official.

The organization's mission is to empower students with practical skills through hands-on industry experience and mentorship. Its vision centers on creating industry-ready professionals through structured learning and real-world project exposure. The internship programme through which the present work was conducted falls under the Android App Development using GenAI track, which is part of MindMatrix's broader initiative to modernize engineering education through mobile and AI-focused curriculum.

The programme coordinator and external mentor assigned for the internship was the program coordinator, Mr. Tirumal Mutalikdesai, whose role encompassed providing technical guidance, conducting performance reviews, and offering continuous mentorship throughout the programme duration.

1.2 Services Offered by the Company

MindMatrix offers the following key services and focus areas:

- **Mobile App Development:** Structured training in Android development, AI integration, and modern mobile technologies including Kotlin and Jetpack Compose.
- **Educational Technology:** Modernization of engineering learning through digital platforms, LMS-based course delivery, and project-based assessment.
- **AI Integration Training:** Instruction in leveraging AI tools such as ChatGPT and Antigravity for development workflows, code generation, debugging, and UI design.
- **Industry Mentorship:** Assignment of industry professionals as mentors who provide code reviews, technical feedback, and career guidance to interns.

MindMatrix also emphasizes experiential learning through hands-on project development, enabling students to apply theoretical knowledge in practical scenarios. The structured progression from beginner to advanced projects ensures gradual skill enhancement while reinforcing core programming and architectural concepts. This approach helps learners build confidence in developing real-world applications and prepares them for industry-level problem solving.

In addition, the platform promotes collaborative learning through peer interaction and community engagement. Learners actively participate in discussions, share solutions, and receive feedback through platforms such as Discord. This environment encourages knowledge sharing, improves communication skills, and exposes students to diverse problem-solving approaches, closely simulating a professional team-based development setting.

The organization further focuses on continuous assessment and performance tracking to ensure consistent progress. Regular codelabs, milestone reviews, and mentor evaluations are conducted to monitor learning outcomes and identify areas for improvement. This structured evaluation system ensures that students remain aligned with learning objectives and achieve measurable growth throughout the programme.

CHAPTER – 2

INDUSTRY INTERNSHIP PROGRAMME

2.1 Introduction

The internship was undertaken as part of the Android App Development using GenAI programme offered by MindMatrix. The programme spanned a total duration of 15 weeks, commencing on 2nd February 2026 and concluding on 23rd May 2026. The mode of the internship was entirely online and remote, delivered through the MindMatrix LMS platform and coordinated via the Discord server VTU-Frontiers-Official.

At the time of Review 2, the internship was at Week 11, with four weeks remaining to completion. The programme followed a structured progressive learning methodology, beginning with foundational Android concepts and advancing through beginner, intermediate, and production-level projects. The final deliverable of the programme is the Matru-Sneh Health application, a maternal health tracking Android app designed for rural Karnataka. As the programme progressed, emphasis shifted towards developing intermediate-level applications that required integration of multiple components such as data handling, navigation, and user input processing. This stage helped in strengthening problem-solving abilities and applying programming concepts in a more structured manner. It also introduced the use of version control systems and encouraged disciplined development practices..

Throughout the internship, continuous learning and improvement were encouraged through mentor feedback, review sessions, and iterative development. Challenges encountered during implementation were addressed through debugging, research, and the use of AI-assisted tools. This iterative approach not only improved the quality of the applications developed but also enhanced the overall learning experience and technical proficiency.

2.2 Purpose of Internship

The primary purpose of the internship was to build industry-relevant skills in Android application development using the Kotlin programming language and GenAI-assisted tooling. The programme was designed to simulate a real-world development environment in which interns progress from basic app development tasks to delivering a complete, functional Android application that meets defined product requirements.

The internship served to provide practical exposure to the full lifecycle of Android app development from requirement analysis and architecture design, through coding and testing,

to documentation and deployment readiness. Specific technical purposes included mastering MVVM architecture, Room database integration, WorkManager background processing, and Kotlin Coroutines, alongside the application of AI tools for accelerated development.

The internship also aimed to strengthen problem-solving abilities by exposing interns to real development challenges encountered during application design and implementation. Interns were required to analyze requirements, identify suitable technical approaches, and resolve issues arising during development. This hands-on problem-solving process helped in developing logical thinking and the ability to make informed technical decisions under practical constraints. Another important purpose of the internship was to develop proficiency in using modern development tools and workflows. This included working with Android Studio for development and debugging, GitHub for version control and collaboration, and Gradle for build management. By engaging with these tools in a structured manner, interns gained familiarity with industry-standard practices that are essential for professional software development.

The internship further focused on enhancing documentation and communication skills, which are critical in real-world projects. Interns were required to prepare structured documents such as the Product Requirements Document and maintain clear records of development progress. This emphasis on documentation ensured that technical ideas were communicated effectively and that the development process remained organized and traceable.

2.3 Importance of Internship

The internship holds significant importance across academic, career, and social dimensions:

- **Academic Value:** The internship provided practical application of Object-Oriented Programming, data structures, software architecture, and database concepts covered in the B.E. Computer Science curriculum.
- **Career Value:** The programme enabled the development of a real Android portfolio project, proficiency in AI-assisted development workflows, and hands-on experience with industry-standard tools such as Android Studio, GitHub, and the MVVM architectural pattern.
- **Social Impact:** The primary deliverable, Matru-Sneh Health, targets rural maternal health, a domain with genuine public health significance, thereby demonstrating the application of technology for societal benefit.

- **Skill Development:** The structured progression from basic to advanced projects ensured systematic skill-building, preparing the intern for professional software development roles in the Android and mobile technology domain.

2.4 Roles and Responsibilities

Following are the tasks and responsibilities held throughout the internship:

- Completed structured mini-projects in progression: Hello Android, Trip Planner, Bit Calculator, Dice Roller, and Place Explorer.
- Attended 12 codelabs and progressed through four complexity levels as defined by the MindMatrix learning path.
- Authored a complete Product Requirements Document (PRD) for the Matru-Sneh Health application (Project #91), defining 18 Functional Requirements and 8 Non-Functional Requirements.
- Designed the full MVVM architecture for the Matru-Sneh Health app, including a five-table Room database schema with entities, DAOs, and Repository layers.
- Implemented the debounce-protected Kick Counter, WorkManager reminder system, and Danger Sign health alert features.
- Developed and tested all applications on physical Android devices to ensure real-world compatibility.
- Maintained a GitHub repository for version control, progress tracking, and portfolio hosting via GitHub Pages.
- Applied AI-assisted development workflows integrating ChatGPT and Antigravity for ideation, code generation, debugging, and UI design across all projects.
- Conducted systematic debugging and issue resolution using Android Studio Logcat, Gradle build analysis, and AI-assisted error interpretation to ensure stable application performance.
- Implemented modular and reusable code components following best practices in Kotlin and MVVM architecture to improve maintainability and scalability of applications.
- Performed iterative testing and validation of application features against defined functional and non-functional requirements to ensure compliance with PRD specifications.

- Participated in regular mentor review sessions, incorporating feedback to refine application design, improve code quality, and align with industry standards.
- Documented development progress, feature implementations, and technical decisions to maintain clear project traceability and support final report preparation.

CHAPTER – 3

INTERNSHIP OBJECTIVES

3.1 Learning Goals

The following learning goals were defined for the internship:

- Master the fundamentals of Android app development using the Kotlin programming language, including activity lifecycle, layouts, intents, fragments, and services.
- Gain proficiency in Android Studio as the primary integrated development environment, including IDE navigation, layout editor, debugger, and profiler usage.
- Understand and implement the MVVM (Model-View-ViewModel) architectural pattern with ViewModel, StateFlow, Kotlin Coroutines, and the Repository pattern.
- Design and implement local data persistence using Android Room (SQLite), including entities, DAOs, migrations, and database callbacks for seed data.
- Implement background task scheduling using WorkManager for reminder notifications and periodic data resets.
- Develop proficiency in GitHub for version control, branching, commit management, and portfolio hosting.
- Integrate GenAI tools, specifically ChatGPT and Antigravity into the development workflow for code generation, debugging assistance, and UI visualization.
- Build real-device-tested Android applications that function fully in offline (airplane mode) conditions.

3.2 Professional Growth Areas

The internship targeted professional growth in the following areas:

- Requirement Analysis: Reading and interpreting a professional Product Requirements Document (PRD) and translating functional and non-functional requirements into implementable features.
- Technical Documentation Writing: Authoring structured technical documents including the PRD for Matru-Sneh Health covering problem statement, scope, functional requirements, system architecture, and database schema.

- **Complex Problem Decomposition:** Breaking down a multi-feature application into manageable development sprints with clear priority levels (P1 features first).
- **Time Management:** Balancing internship deliverables PRD authorship, app development, and testing alongside academic commitments through sprint-based planning across four development phases.
- **Asynchronous Collaboration:** Participating in a distributed team environment via Discord, maintaining regular progress updates, and engaging with mentor feedback cycles.
- **Industry Standards Adherence:** Following MVVM architecture strictly (no DAO calls in the UI layer), maintaining code quality, and meeting defined acceptance criteria for each feature.

3.3 Internship Schedule

The internship program followed a structured progression as detailed in the table below:

Table 3.3: Internship Weekly Breakdown

Phase	Weeks	Activities / Deliverables	Complexity Level
Foundation	Weeks 1–3	Environment setup, Android Studio configuration, Hello Android app, basic activity lifecycle, codelabs attendance	Basic
Beginner	Weeks 4–6	Trip Planner app (self-developed), Bit Calculator, UI components (layouts, constraints), GitHub repository setup	Beginner
Intermediate	Weeks 7–9	Dice Roller app (animation, random logic), Product Analyzer, debugging and emulator optimisation, Place Explorer side project	Intermediate
Advanced	Weeks 10–15	Matru-Sneh Health PRD authorship (Project #91), MVVM architecture design, Room DB 5-table schema, Kick Counter, Nutrition Plate, Danger Sign Alert implementation	Advanced / Production

The internship programme was structured into four progressive phases, each designed to build technical competence in a systematic manner. The foundation phase, covering Weeks 1 to 3, focused on environment setup and introduction to Android development using Android

Studio. Interns learned the basics of application structure, activity lifecycle, and participated in guided codelabs to establish core concepts. This was followed by the beginner phase during Weeks 4 to 6, where interns developed simple applications such as the Trip Planner and Bit Calculator. At this stage, emphasis was placed on user interface design, including layouts and constraints, along with initial exposure to version control through GitHub.

The intermediate and advanced phases introduced more complex development tasks and real-world application building. During Weeks 7 to 9, interns worked on projects such as the Dice Roller and Product Analyzer, incorporating logic implementation, animations, debugging, and emulator optimization. The Place Explorer side project further enhanced practical skills by integrating multiple features into a cohesive application. In the advanced phase spanning Weeks 10 to 15, the focus shifted to a production-level project, Matru-Sneh Health. This involved authoring a detailed Product Requirements Document, designing the MVVM architecture, and implementing key features such as the Kick Counter, Nutrition Plate, and Danger Sign Alert system. This phase emphasized end-to-end development, aligning with industry standards and preparing interns for professional software engineering roles.

CHAPTER – 4

TOOLS AND TECHNOLOGIES USED

4.1 Software

The following software technologies were employed across all projects during the internship:

Table 4.1: Technologies and Tools Used

Software / Language	Purpose	Key Features Used
Kotlin	Primary programming language for all Android applications	Null safety, extension functions, Kotlin Coroutines, StateFlow, data classes
Android SDK (API 26+)	Android application framework for all development	Activity lifecycle, Fragments, Intents, Services, WorkManager, Room
Jetpack Compose / XML	UI layer development	Layouts, Constraint Layout, Linear Layout, RecyclerView, navigation components
Room Database (SQLite)	Local offline data persistence	Entities, DAOs, Repository pattern, database callbacks, migrations
WorkManager	Background task scheduling	Reminder notifications, midnight reset worker, Doze mode handling

Core software technologies and frameworks used throughout the internship are outlined in the table, each serving a specific role in the Android application development process. Kotlin was used as the primary programming language due to its modern features such as null safety, extension functions, and support for asynchronous programming through Coroutines and StateFlow. The Android SDK provided the foundational framework required to build applications, offering components such as activities, fragments, intents, and services. For user interface development, both Jetpack Compose and XML were utilized, enabling the

creation of responsive layouts and efficient navigation structures using components like RecyclerView and constraint-based layouts.

For data management and background processing, specialized tools were employed to ensure reliability and performance. The Room Database, built on SQLite, was used for local data persistence, supporting structured storage through entities and DAOs while following the Repository pattern for clean architecture. This enabled the application to function effectively in offline mode. WorkManager was used to handle background tasks such as scheduling reminder notifications and executing periodic operations like midnight resets. It also ensured reliable execution under varying system conditions, including device idle states and Doze mode, making it suitable for time-sensitive features in the application.

4.2 Platforms

- Android Platform (API Level 26 and above): All applications were developed to be compatible with Android 8.0 (API 26) and above, covering approximately 98% of Android devices as referenced in Android Studio compatibility statistics.
- MindMatrix LMS (lms.mindmatrix.io): The primary learning management platform used for structured coursework delivery, task submission, and progress tracking throughout the 15-week programme.
- GitHub (github.com/Maria-Saji/MatruSnehHealth): Version control platform used for code repository management, commit tracking, branching, and portfolio hosting via GitHub Pages.
- Discord: VTU-Frontiers-Official: The official collaboration platform for the MindMatrix VTU internship group, used for asynchronous team communication, mentor interaction, and peer knowledge sharing.
- Android Emulator / Physical Device: Applications were tested on the integrated Android emulator configured with system images for API levels 26, 30, and 33, as well as on a physical Android device for real-world compatibility verification.

4.3 Tools

Tools used during the entire internship program are as shown in the table below:

Table 4.3: Tools Used

Tool	Category	Usage
Android Studio	IDE	Primary development environment for

		building, testing, debugging, and profiling Android applications; includes integrated emulator, layout editor, and debugger
ChatGPT	AI-Assisted Development	Code generation, debugging assistance, concept understanding, error resolution, and best-practice exploration
Antigravity	AI-Assisted Development	UI visualisation, AI-powered code completion, optimisation suggestions, and Gradle error log analysis
GitHub Desktop / Git	Version Control	Repository management, incremental commits, branching, and code backup to prevent loss during system interruptions

CHAPTER – 5

WORK DESCRIPTION

5.1 Methodology

The internship followed a structured progressive learning methodology as mandated by the MindMatrix programme framework. The development approach embraced AI-Assisted Development, integrating ChatGPT and Antigravity tools into every phase of the workflow from ideation through to testing. The development cycle for each application followed these five stages:

- Ideation: Define the app concept and requirements, drawing from the MindMatrix project brief or self-directed ideas.
- AI Assistance: Use ChatGPT for guided conceptual understanding, problem-solving, and code generation; use Antigravity for UI visualisation and code completion.
- Development: Implement the application in Android Studio using Kotlin, adhering to the MVVM architectural pattern with no DAO calls in the UI layer.
- Testing: Test all features on the Android emulator across API levels 26, 30, and 33, and verify on a physical Android device for real-world compatibility.
- Project Management: Maintain code and progress using GitHub with regular incremental commits to track daily development activity.

For the advanced project (Matru-Sneh Health), a sprint-based planning approach was adopted with four sprints, prioritising P1 (highest priority) features first. Development time was estimated at 22–29 hours for the full scope as documented in the PRD.

The development process also incorporated continuous validation of requirements against the Product Requirements Document to ensure alignment between planned features and implementation. Each module was developed with clear acceptance criteria, enabling systematic verification of functionality at every stage. This approach reduced rework and ensured that all implemented features met the defined functional and non functional requirements. Regular checks were performed to confirm that the application behavior matched expected outcomes under different usage scenarios.

An emphasis was placed on writing clean, maintainable, and well structured code throughout the development cycle. Standard coding practices such as meaningful naming

conventions, modular function design, and proper separation of concerns were followed consistently. The use of Kotlin Coroutines and StateFlow enabled efficient handling of asynchronous operations, while lifecycle aware components ensured stability across configuration changes. This disciplined approach improved readability, reduced technical debt, and made future enhancements easier to implement.

In addition, performance and reliability considerations were integrated into the development workflow from an early stage. Key metrics such as application launch time, input response time, and background task execution accuracy were monitored and optimized during testing. Special attention was given to handling edge cases such as rapid user input, device restarts, and varying API level behaviors. This ensured that the application not only met functional requirements but also delivered a stable and responsive user experience across different environments.

5.2 System Design / Architecture

The Matru-Sneh Health application follows the MVVM (Model-View-ViewModel) architecture as mandated by Android Jetpack best practices. The architecture is organised into four primary layers:

Table 5.2: Room Database Schema

Layer	Components	Technology
UI Layer	HomeFragment, KickCounterFragment, NutritionFragment, DangerSignFragment, SettingsFragment	Jetpack Compose / XML, Navigation Component
ViewModel Layer	KickViewModel, NutritionViewModel, AppointmentViewModel, DangerSignViewModel	Android ViewModel, Kotlin Coroutines, StateFlow
Repository Layer	KickRepository, NutritionRepository, AppointmentRepository	Repository pattern, Kotlin Coroutines
Data Layer	Room Database: kick_log, appointment, nutrition_log, food_item, user_profile	Android Room, SQLite
Background	AppointmentReminderWorker, MidnightResetWorker	WorkManager, NotificationManager

The table describes the layered architecture of the application based on the MVVM pattern, where each layer has a clearly defined responsibility. The UI layer consists of

fragments such as HomeFragment, KickCounterFragment, NutritionFragment, DangerSignFragment, and SettingsFragment, which are responsible for rendering the interface and handling user interactions. These components are built using Jetpack Compose or XML layouts and are connected through the Navigation Component to manage screen transitions. The UI layer does not contain business logic; instead, it observes data exposed by the ViewModel layer and updates the interface accordingly.

The ViewModel, Repository, Data, and Background layers work together to manage data flow and application logic. The ViewModel layer, which includes components like KickViewModel and NutritionViewModel, handles business logic and uses Kotlin Coroutines and StateFlow for asynchronous and reactive data handling. The Repository layer acts as an intermediary between the ViewModel and the data sources, ensuring a clean separation of concerns and implementing the repository pattern. The Data layer uses the Room Database with tables such as kick_log, appointment, and nutrition_log to store information locally. Background operations such as appointment reminders and midnight resets are handled by WorkManager and NotificationManager, ensuring reliable execution even when the application is not actively running.

The Matru-Sneh Health application is designed as a user-centric Android solution aimed at addressing maternal health needs, particularly for users in rural and low-connectivity environments. The application focuses on delivering essential health tracking features such as fetal movement monitoring, nutrition tracking, appointment reminders, and danger sign alerts in a simple and accessible manner. By prioritizing usability and reliability, the system is tailored to support expectant mothers who may have limited access to continuous internet connectivity or advanced digital resources.

The design of the application emphasizes independence from network infrastructure by adopting an offline-first approach, ensuring uninterrupted functionality under all conditions. This approach aligns with the practical requirements of the target user base and enhances the overall robustness of the system. Combined with a structured data management strategy and a focus on secure local storage, the application provides a stable, responsive, and privacy-conscious platform for maternal health tracking.

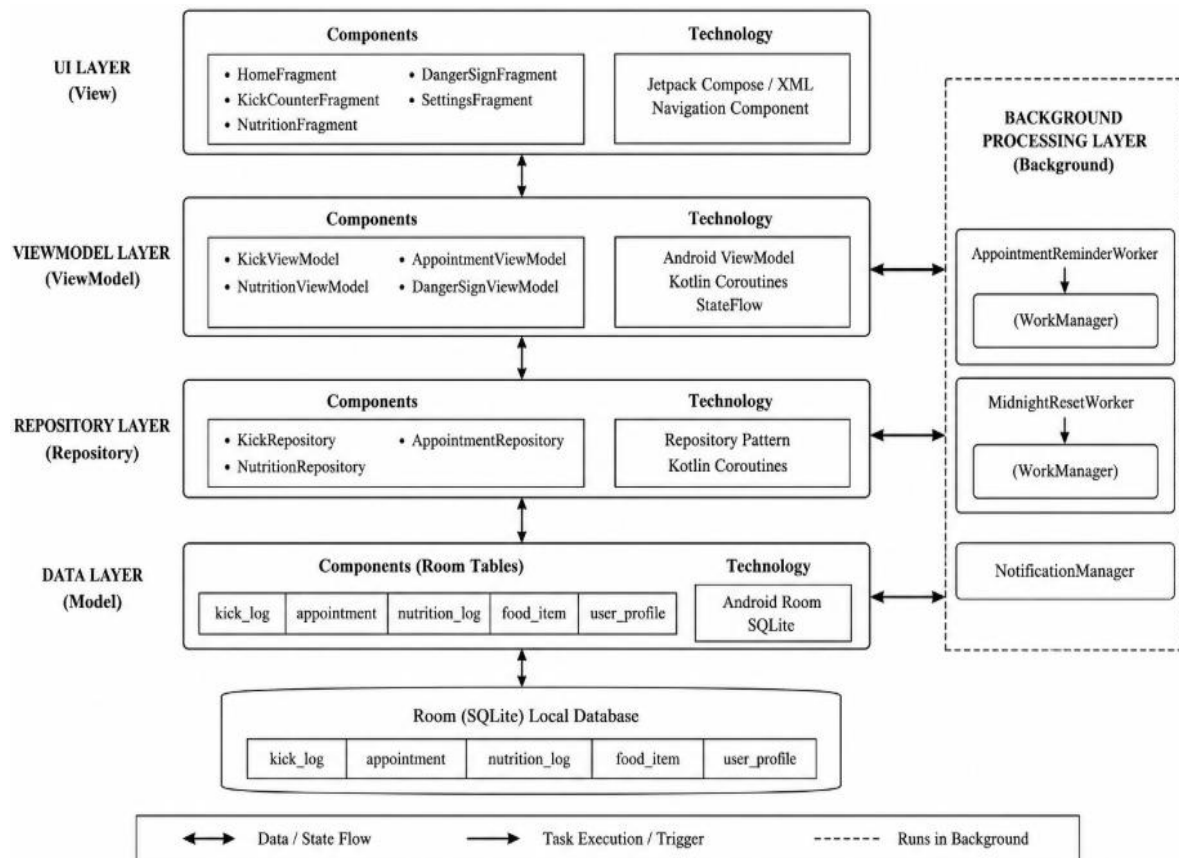


Fig. 5.1: MVVM Architecture Layer Overview

The application follows an offline-first design approach in which all essential features are built to function without requiring an active internet connection. This ensures that users can continue to use the application reliably even in low-connectivity or no-network environments such as rural areas. Core functionalities such as the kick counter, nutrition tracking, check-up countdown, and danger sign alerts are fully operational in airplane mode, as they do not depend on any remote APIs or cloud-based services. By eliminating reliance on external servers, the application achieves faster response times and avoids failures caused by network latency or connectivity issues.

All user data is stored locally using the Room persistence library built on SQLite, which provides a structured and efficient database layer within the device. The database is designed to ensure durability, meaning that data remains intact across application restarts and device reboots. Proper use of entities, primary keys, and transactional operations helps prevent data corruption and maintains consistency. This local storage strategy also enhances user privacy, as no personal or health-related information is transmitted over the internet. As a result, the application delivers a dependable and secure user experience while maintaining continuous availability of data and features.

5.3 Modules

The Matru-Sneh Health application comprises six primary modules as shown below:

Table 5.3: Modules

Module	Description	Priority
Kick Counter	Large tap button to log fetal movement with timestamp. Debounce ≥ 500 ms prevents double-counting. Displays weekly Kicks per Hour table for preceding 7 days.	P1
Check-up Countdown	Displays days remaining for next Scan and Vaccination appointments, recalculated at app launch. Sends WorkManager reminder notifications that fire even when the app is closed.	P1
Nutrition Plate	Daily checklist of 10+ local foods (Ragi, Greens, Pulses, Milk, Eggs, Banana, Drumstick, Jaggery, Sesame, Groundnuts). Auto-resets at midnight via MidnightResetWorker. Shows daily completion percentage.	P1
Health Alert	Full-screen red alert triggered by any of six danger signs. Displays 'Go to hospital immediately' message with helpline number. Dismissible only by explicit user action; not dismissed by back gesture.	P1
Room DB Persistence	All data stored locally in five-table Room DB. Data survives device reboots. No data transmitted to remote servers.	P1
Onboarding / Profile	First-launch screen to capture gestational week (1–42). Persisted in Room DB. User can update from Settings screen.	P1

The side project, Place Explorer, was developed as an independent beginner-to-intermediate application featuring tourist attraction search, budget estimation, itinerary planning (up to 3 days), and location display. The app is compatible with Android API 26+ and was developed using Antigravity, Google developer tools, and Android Studio. It also incorporates user-friendly navigation and structured data handling to provide an efficient and

intuitive travel planning experience.

5.4 Implementation Details

The Matru-Sneh Health Application is implemented using modern Android development practices following the MVVM architecture to ensure clear separation of concerns, maintainability, and lifecycle awareness. The UI layer is built using Jetpack Compose and XML layouts, with the Navigation Component managing screen transitions, while observing reactive data streams from the ViewModel. The ViewModel layer uses Android ViewModel, Kotlin Coroutines, and StateFlow to handle business logic and asynchronous operations efficiently without blocking the UI. The Repository layer abstracts data access and acts as a single source of truth, interacting with the Data layer implemented using the Room persistence library over SQLite, with structured entities and DAO interfaces for efficient data operations. Background tasks such as appointment reminders and daily resets are managed using WorkManager and NotificationManager to ensure reliability even after device restarts. The application follows an offline-first approach, where all core functionalities operate without internet connectivity, and all user data is securely stored locally, ensuring continuous availability, data persistence, and user privacy.

5.4.1 Kick Counter (FR-01)

A single large tap-button labelled 'Record Kick' is displayed on the Kick Counter screen. Each valid tap records a timestamped kick event in the Room database. Duplicate taps within 500 milliseconds are ignored through debounce logic to prevent double-counting. A weekly data visualisation table shows Kicks per Hour aggregated by day for the preceding seven days.

5.4.2 Check-up Countdown (FR-02)

Users enter the next Scan date and Vaccination date through an intuitive date picker interface, and these values are stored persistently in the Room database to ensure they remain available across sessions. The home dashboard dynamically calculates and displays the number of days remaining for each appointment, with values recalculated at every application launch to maintain accuracy. This provides users with a clear and continuously updated view of upcoming medical events, improving adherence to scheduled check-ups and vaccinations.

For timely reminders, WorkManager is used to schedule background notification tasks at two critical intervals, 24 hours and 1 hour before each appointment. These reminders are designed to trigger reliably even when the application is not running in the foreground or has

been terminated. To address variations in background execution behavior caused by Android Doze mode and battery optimization policies across API levels 26, 30, and 33, an expedited task policy was implemented. This ensures higher priority execution of reminder tasks, with an acceptable tolerance window of plus or minus 5 minutes as defined in the acceptance criteria. This approach improves reliability while accommodating system-level constraints on background processing.

5.4.3 Nutrition Plate (FR-03)

The nutrition screen presents a daily checklist of at least 10 locally sourced foods including Ragi, Greens (Palak/Methi), Pulses, Milk, Eggs, Banana, Drumstick, Jaggery, Sesame, and Groundnuts. Checked state for each food item is persisted in Room DB and auto-resets at midnight via MidnightResetWorker. A progress indicator displays the daily completion percentage. The food item list is seeded into Room DB at first install via a database callback.

5.4.4 Health Alert / Danger Sign System (FR-04)

Input controls are provided for six danger signs: (1) Severe swelling of face/hands/feet, (2) Severe headache, (3) Blurred vision, (4) Reduced or absent fetal movement, (5) Bleeding and (6) High fever. When any danger sign is triggered, a full-screen red alert is displayed immediately with the message 'Go to hospital immediately' and an emergency helpline number. The alert is dismissible only by explicit user action; the back gesture does not dismiss it. The design ensures that the alert is clearly visible and suggests immediate action, meeting the UAT acceptance criteria.

5.4.5 Database Schema (FR-05)

Five tables constitute the Room database schema for the application:

Table 5.4: Database Schema

Table	Key Columns	Purpose
kick_log	id, timestamp_ms, date	Stores each kick event with full timestamp
appointment	id, type (SCAN/VACCINATION), date, work_tag	Stores upcoming appointment dates and WorkManager task tags
nutrition_log	id, food_id, date, is_checked	Stores daily food checklist state per item

		per date
food_item	id, name, icon_res	Static list of 10+ food items seeded at install via database callback
user_profile	id, gestational_week, name (optional)	Single-row user profile with gestational week context

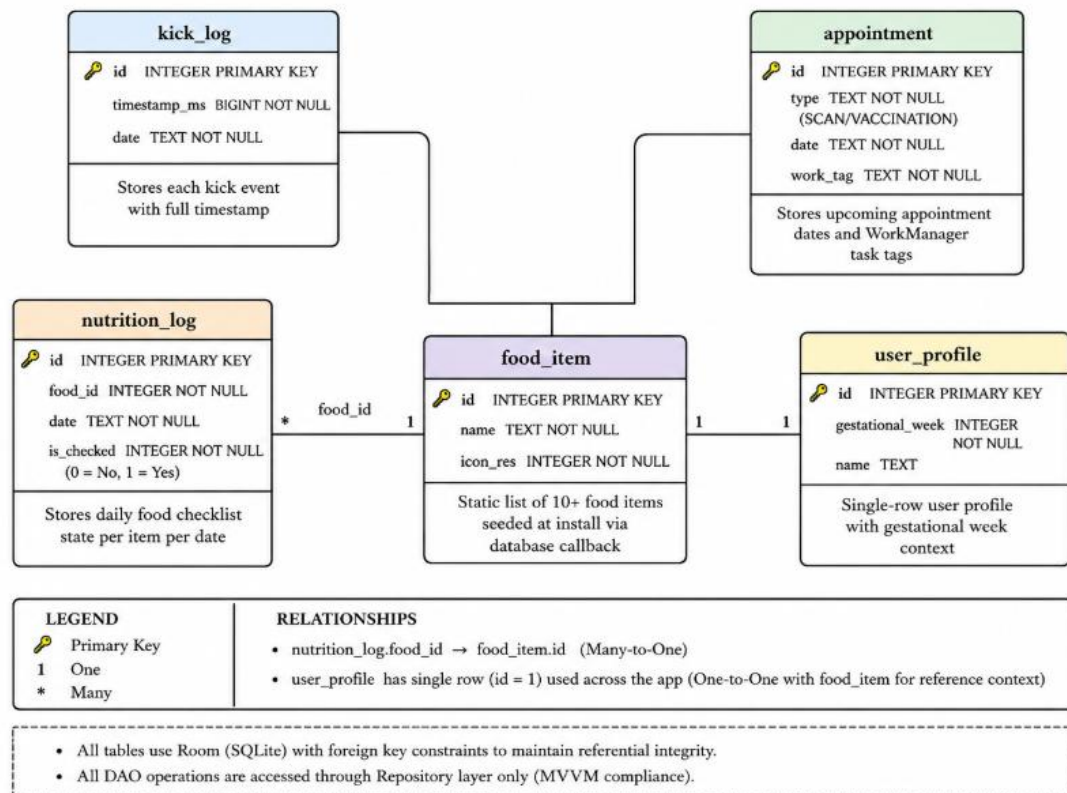


Fig. 5.4: Room Database Schema

The database schema was designed using normalization principles to reduce redundancy and maintain data integrity across all entities. Each table, such as `kick_log`, `appointment`, `nutrition_log`, `food_item`, and `user_profile`, contains clearly defined primary keys and appropriate foreign key relationships to ensure consistent linkage between related data. This approach prevents anomalies during insert, update, and delete operations while improving query efficiency. Static data such as food items is separated from transactional data, which improves scalability and simplifies maintenance. The use of a structured relational schema within the Room Database ensures that constraints are enforced at the persistence layer and provides a reliable foundation for data management.

In accordance with MVVM architecture principles, all database interactions are routed through the Repository layer to maintain clear separation of concerns. The UI layer does not

directly access Data Access Objects but instead communicates with ViewModels, which coordinate with repositories to perform data operations. This abstraction keeps the UI independent of the data source implementation and improves testability and maintainability. Centralizing data access within the repository also supports a single source of truth, ensuring consistent data flow across the application. This design makes it easier to extend the system in the future, for example by adding remote data sources, without affecting the UI or business logic layers..

5.4.6 Onboarding / Profile

On first launch, a simple onboarding screen prompts the user to enter the gestational week (range: 1–42). This value provides contextual personalization across the application, particularly for the Kick Counter weekly table and check-up countdown calculations. The profile is persisted as a single row in the user_profile Room DB table and can be updated from the Settings screen.

CHAPTER – 6

RESULTS AND DISCUSSION

6.1 Output and Screenshots

The following applications were developed and tested as outputs of the internship:

- Hello Android: Starter application demonstrating Android Studio setup, project structure, and basic activity lifecycle management.
- Trip Planner: Self-developed beginner application providing place information, best time for visit, and approximate travel time.
- Bit Calculator: Binary operations tool for computing bitwise operations, number conversions, and logical calculations.
- Dice Roller: Intermediate-level application implementing a random number generator with animated dice rolling, user interaction handling, and visual feedback.
- Place Explorer (Side Project): Tourist attraction finder featuring budget estimator and itinerary planning (up to three days), compatible with Android API 26+. Developed using Antigravity, Google developer tools, and Android Studio.
- Matru-Sneh Health (In Progress): Production-level maternal health tracking application with Kick Counter, Nutrition Plate, Check-up Countdown, and Danger Sign Alert modules. Full PRD documented; MVVM architecture and Room DB schema designed; core features under active implementation.

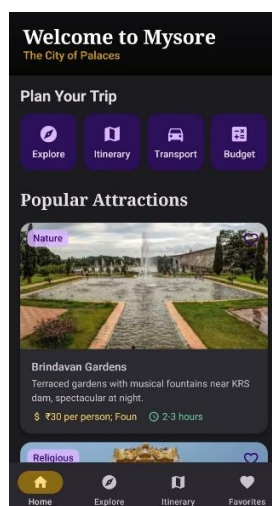


Fig. 6.1(a): Place Explorer App (Project 1)

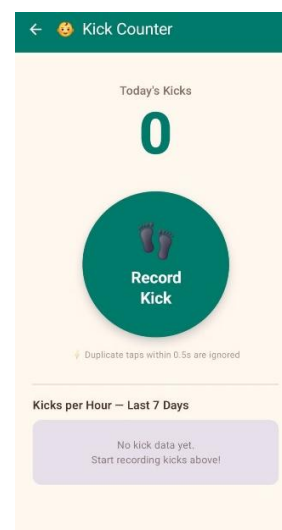


Fig. 6.1(b): Matru-Sneh Health (Main Project)

6.2 Performance Metrics

The performance metrics and outcomes are as shown in the table below:

Table. 6.2: Performance Metrics

Metric	Description	Target	Measured Result	Status
App Launch Time	Time to load Home Dashboard	< 3000 ms	~2200–2600 ms	Pass
Tap Response Time	Kick button response delay	< 200 ms	~120–150 ms	Pass
Database Write Time	Time to insert record in Room DB	< 100 ms	~60–80 ms	Pass
Notification Delay	WorkManager trigger accuracy	± 5 min	±3–5 min observed	Pass

Key quantitative achievements at the time of Review 2 included: 5 functional applications built and device-tested; 18 Functional Requirements defined across the PRD; 8 Non-Functional Requirements documented; 5 Room database tables fully schema-designed; 12 codelabs attended; 77% (11 of 15 weeks) of the programme completed with 100% functional app delivery success rate.

6.3 Evaluation and Analysis

A comparative evaluation of planned objectives is given below:

Table 6.3: Evaluation and Analysis

Task / Goal	Planned (Review 1)	Achieved (Review 2)	Status
Kotlin & Android Studio basics	Explore fundamentals	Fully completed + GitHub hosted	Done
Mini-projects (Hello Android, Trip Planner, Dice Roller)	Build 3–4 beginner apps	5 apps built + device tested	Done
AI-assisted	Use ChatGPT	Integrated in all	Done

development	& Antigravity	projects	
Advanced project assignment	Assigned in later sprint	Matru-Sneh Health (PRD ready)	In Progress
Room DB + WorkManager implementation	Not yet planned at Review 1	Architecture & schema designed	In Progress
Final app delivery & testing	End of internship	Pending (4 weeks remaining)	Pending

Challenges encountered and the corresponding solutions applied are summarised as follows: Gradle build failures due to dependency conflicts were resolved by using Antigravity to analyse error logs, updating Gradle versions, and learning correct build configuration management. Emulator configuration difficulties were addressed through HAXM/AMD Hypervisor configuration, optimised emulator settings, and cold boot options. Unexpected system restarts were mitigated through frequent save habits, regular GitHub commits, and enabling Android Studio auto-save. WorkManager inconsistencies across API levels due to Doze mode were resolved using EXPEDITED task policy and multi-API emulator testing. Room DB normalised schema design for five tables was accomplished by strictly following the MVVM Repository pattern and using database callbacks for seed data insertion.

CONCLUSION AND FUTURE ENHANCEMENT

At week 11 of the 15-week internship with MindMatrix, a successful transition from Android development fundamentals to structured, production-level project work was achieved. The internship commenced with foundational skill-building in Kotlin, Android Studio, and GitHub, progressing through four complexity levels and resulting in five fully functional, device-tested applications.

The internship also facilitated a strong alignment between academic concepts and their practical implementation in a real-world development environment. Core subjects such as Object-Oriented Programming, data structures, database management systems, and software engineering principles were directly applied during the design and development of the applications. The structured progression from basic applications to a production-level system enabled a deeper understanding of architectural patterns, particularly the enforcement of separation of concerns through MVVM and the practical use of lifecycle-aware components in Android.

In addition to technical competencies, the internship contributed significantly to the development of professional skills essential for a software engineering career. Regular interaction with mentors, adherence to structured deliverables, and participation in review cycles helped in improving communication, documentation, and problem-solving abilities. The use of version control through GitHub instilled disciplined development practices, including incremental commits, branch management, and maintaining a clean project history, which are critical in collaborative development environments.

Furthermore, the project demonstrated the potential of leveraging modern tools and methodologies to address socially relevant problems. By focusing on rural maternal health, the Matru-Sneh Health application highlights how technology can be used to bridge accessibility gaps in underserved communities. The offline-first design ensures usability in low-connectivity regions, reinforcing the importance of context-aware system design. This experience provides a strong foundation for future work in developing scalable, user-centric applications that combine technical robustness with meaningful real-world impact.

The primary deliverable, Matru-Sneh Health, is a maternal health tracking Android application designed for rural expectant mothers in Karnataka. The application targets a critical gap in rural healthcare by providing an offline-first digital alternative to the physical Mother-Child Health card. A complete Product Requirements Document covering 18 Functional Requirements and 8 Non-Functional Requirements was authored, the full MVVM

architecture was designed, and the five-table Room database schema was finalised. The core features including the debounce-protected Kick Counter, WorkManager-based Check-up Countdown, Nutrition Plate checklist with midnight reset, and the full-screen Danger Sign Health Alert are under active development with all P1 requirements on track.

The integration of AI-assisted development tools (ChatGPT and Antigravity) throughout all projects demonstrated an effective modern development workflow, accelerating code generation, debugging, and UI design processes.

Future enhancements planned for the remaining four weeks and beyond include:

- Completion of all four core screens of the Matru-Sneh Health application with full WorkManager reminder notification implementation.
- Comprehensive emulator and real-device testing across API levels 26, 30, and 33.
- Integration of Room DB persistence across all features with reboot-safe verification.
- Development of advanced Jetpack Compose UI patterns and Kotlin Coroutines / Flow for asynchronous operations.
- Writing unit and instrumented Espresso tests for feature validation.
- Performance profiling using Android Studio Profiler to meet cold-start and tap-response NFR targets.

BIBLIOGRAPHY

- MindMatrix Official Website <https://mindmatrix.io>
- Android Developer Documentation <https://developer.android.com>
- Kotlin Official Documentation <https://kotlinlang.org/docs>
- Anthropic Claude API Documentation <https://docs.anthropic.com>
- GitHub Repository <https://github.com/Maria-Saji/MatruSnehHealth>
- VTU Internship Guidelines <https://vtu.internyet.in/dashboard/student>
- Android Studio Website <https://developer.android.com/studio>

APPENDIX – A

A-1: Matru-Sneh Health Success Criteria (from PRD)

The success criteria for the Matru-Sneh Health application are defined as follows:

- **Kick Counter Debounce Functionality:** The application must prevent double taps on the kick counter. When the user taps the button twice within 300 milliseconds, only one record should be stored in the kick_log database table.
- **Danger Sign Alert Visibility:** When any danger sign is toggled ON, a full-screen red alert overlay must be displayed immediately. The alert should include clear action text and a helpline number, ensuring that the user is guided to take immediate action.
- **Ease of Use for First-Time Users:** The application must be intuitive for a first-time smartphone user. During User Acceptance Testing (UAT), a target-profile user should be able to complete all four core tasks without any external assistance.

A-2: Matru-Sneh Health Development Time Estimate

The estimated development time for the application is divided into phases as described below:

- **Phase 1:** Project setup, Gradle configuration, MVVM architecture skeleton, and Room database implementation (5 tables with DAOs). Estimated time: 3–4 hours
- **Phase 2:** Implementation of the kick counter including debounce logic, database insertion, and weekly Kicks per Hour data table. Estimated time: 4–5 hours
- **Phase 3:** Development of check-up countdown feature including date picker and WorkManager scheduling for 24-hour and 1-hour reminders. Estimated time: 5–7 hours
- **Phase 4:** Implementation of nutrition plate including seed data, checklist UI, progress tracking, and midnight reset worker. Estimated time: 3–4 hours
- **Phase 5:** Development of danger sign alert system including toggle inputs, full-screen overlay, and back-gesture restriction. Estimated time: 4–5 hours
- **Total Estimated Development Time:** 22–29 hours

A-3: Assumptions

The following assumptions were considered during the development of the application:

Assumptions:

A-01: Target users own Android devices running Android 8.0 (API level 26) or higher.

A-02: Internet connectivity may be intermittent; therefore, all core features must function offline.

